

Algorithms (Formally)

An **algorithm** is a *finite* sequence of **well-defined** steps to perform a task such that

- each step is a clear instruction that can be done in a finite amount of time
- the sequence in which the steps are done is clear
- the process is *guaranteed* to stop after a *finite* number of steps have been done

While our initial definition of algorithms

a list of instructions

is *correct*, when talking about programming, we require more detail

Pseudocode

In order to make algorithms easier to understand, one way of writing them is through *pseudocode*

pseudo meaning fake

And this allows us to think of the **structure** of the algorithm without worrying about the details of a particular *language*

Pseudocode

An algorithm that finds the volume of a cube with side length l

```
1. Input  $l$  ( $l$  is the length of one side of the cube)
2.  $\text{volume} \leftarrow l^3$ 
3. output volume
```

By definition, pseudocode doesn't have a format, but notice that

1. the steps are numbered so it's easy to reference
2. there are comments explaining things
3. the $\leftarrow$ means *assignment*, we're calculating l^3 , and assigning it to the name volume

Control structures

From scratch you've been given some exposure to things called *control structures*

They are used to tel the algorithm what to do at *different times*

There are two types

- conditional controls
- loop controls

For our case, we'll use

and

- if-then
- if-then-else
- for-do
- while-do
- repeat-until

If structures

Consider the following

1. input x (x is a student's grade as a percent)
2. If $x \geq 60$ then ($\geq$ means greater than or equal)
 - 2.1 output "passed"

If structures

Consider the following

1. input x (x is a student's grade as a percent)
2. If $x \geq 60$ then ($\geq$ means greater than or equal)
 - 2.1 output "passed"
 - else
 - 2.2 output "failed"

If structures

It's also possible to *nest* if structures, meaning we put an if else statement *inside* another if statement

```
1. Input x (x is a student's grade as a percent)
2. if x ≥ 90 then
    2.1 output "A"
    else
3. if x ≥ 80 then
    3.1 output "B"
    else
4. if x ≥ 70 then
    4.1 output "C"
    else
5. if x ≥ 60 then
    5.1 output "D"
    else
    5.2 output "F"
```

If structures

It's also possible to *nest* if structures, meaning we put an if else statement *inside* another if statement

```
1. Input 63 (x is a student's grade as a percent)
2. if 63 ≥ 90 then
    2.1 output "A"
    else
3. if 63 ≥ 80 then
    3.1 output "B"
    else
4. if 63 ≥ 70 then
    4.1 output "C"
    else
5. if 63 ≥ 60 then
    5.1 output "D"
    else
    5.2 output "F"
```


If structures

It's also possible to *nest* if structures, meaning we put an if else statement *inside* another if statement

```
1. Input 63 (x is a student's grade as a percent)
2. if 63 ≥ 90 then
    2.1 output "A"
    else
3. if 63 ≥ 80 then
    3.1 output "B"
    else
4. if 63 ≥ 70 then
    4.1 output "C"
    else
5. if 63 ≥ 60 then
    5.1 output "D"
    else
    5.2 output "F"
```

If structures

It's also possible to *nest* if structures, meaning we put an if else statement *inside* another if statement

```
1. Input 63 (x is a student's grade as a percent)
2. if 63 ≥ 90 then
    2.1 output "A"
    else
3. if 63 ≥ 80 then
    3.1 output "B"
    else
4. if 63 ≥ 70 then
    4.1 output "C"
    else
5. if 63 ≥ 60 then
    5.1 output "D"
    else
    5.2 output "F"
```

If structures

It's also possible to *nest* if structures, meaning we put an if else statement *inside* another if statement

```
1. Input 63 (x is a student's grade as a percent)
2. if 63 ≥ 90 then
    2.1 output "A"
    else
3. if 63 ≥ 80 then
    3.1 output "B"
    else
4. if 63 ≥ 70 then
    4.1 output "C"
    else
5. if 63 ≥ 60 then
    5.1 output "D"
    else
    5.2 output "F"
```

If structures

It's also possible to *nest* if structures, meaning we put an if else statement *inside* another if statement

```
1. Input 63 (x is a student's grade as a percent)
2. if 63 ≥ 90 then
    2.1 output "A"
    else
3. if 63 ≥ 80 then
    3.1 output "B"
    else
4. if 63 ≥ 70 then
    4.1 output "C"
    else
5. if 63 ≥ 60 then
    5.1 output "D"
    else
    5.2 output "F"
```

If structures

It's also possible to *nest* if structures, meaning we put an if else statement *inside* another if statement

```
1. Input 63 (x is a student's grade as a percent)
2. if 63 ≥ 90 then
    2.1 output "A"
    else
3. if 63 ≥ 80 then
    3.1 output "B"
    else
4. if 63 ≥ 70 then
    4.1 output "C"
    else
5. if 63 ≥ 60 then
    5.1 output "D"
    else
    5.2 output "F"
```

If structures

It's also possible to *nest* if structures, meaning we put an if else statement *inside* another if statement

```
1. Input x (x is a student's grade as a percent)
2. if x ≥ 90 then
    2.1 output "A"
    else
3. if x ≥ 80 then
    3.1 output "B"
    else
4. if x ≥ 70 then
    4.1 output "C"
    else
5. if x ≥ 60 then
    5.1 output "D"
    else
    5.2 output "F"
```

For

Loop controls tell you how many times a step in the algorithm should be executed

an algorithm that finds the smallest number in a list of numbers

```
1. input the number of vlaues in list, n
2. input the list of numbers x_1, x_2,, ... ,x_n
3. min ← x_1
4. for "numbers starting at 2 and ending at n" do
    4.1 if x_i < min then
        4.1.1 min ← x_i
5. output min
```

For

Loop controls tell you how many times a step in the algorithm should be executed

an algorithm that finds the smallest number in a list of numbers

```
1. input the number of vlaues in list, n
2. input the list of numbers x_1, x_2,, ... ,x_n
3. min ← x_1
4. for i=2 to n do
    4.1 if x_i < min then
        4.1.1 min ← x_i
5. output min
```